# 1 The RTCU Platform

## 1.1 The RTCU Platform

The RTCU platform consists of a powerful programming environment and a broad range of RTCU devices, which offers the possibility of both digital- and analog I/O combined with wireless technologies such as Cellular, Bluetooth, WiFi and GPS. It is through these combinations that it becomes possible to solve many different tracking-, control-, regulation- and surveillance-applications.

With the RTCU platform, all necessary functions are combined in only one product, which is very easy to program in the text-based programming language "VPL". This combination opens the door to a wide array of efficient custom-tailored solutions today, and yet the same product is also able to solve tomorrow's challenges just through simple modifications to the program.

It is not necessary, however, to buy expensive customised adjustments of the product, because it is very easy to learn how to write a program that solves a specific problem on a standard PC by using the supplied development environment called RTCU IDE.

One of the major applications of the RTCU platform is remote surveillance thanks to its built-in cellular module. This module makes it possible to construct low-priced autonomous systems, which will be able to control and/or supervise different processes on distant localities without access to the wire-based telephone lines (PSTN). The system can be implemented in such a way that during normal conditions, it is able to manage control and surveillance all by itself. If an alarm should occur, the system is able to deliver an SMS-message, send a notification over a TCP/IP network, send a voice-message or send an e-mail to a predefined receiver.

The RTCU platform supports many IP based protocols, such as: The RTCU Communication Hub, sockets, MQTT, SMTP and FTP. TLS is fully supported.

A good place to start is:

- RTCU IDE Integrated Development Environment
- Quick start guide
- Examples
- Tutorial

For further and more detailed information please refer to the RTCU Execution Architecture.


© 2025 Logic IO, www.logicio.com

## 1.2 RTCU Execution Architecture

### 1.2.1 RTCU Execution Architecture

*The **RTCU Execution Architecture** defines the functionality, capacity and performance available to the application.*

Starting in **1999** the **first generation** Execution Architecture were introduced initially without a formal name, but later known simply as the **'SMALL'** architecture. The hardware supported was with a modest 16 bit processor with only 10 KB of RAM.

In **2002** the **second generation** Execution Architecture was introduced at the same time as a full TCP/IP stack allowing GPRS support in one of the first devices in the world - the RTCU M10. This generation was named the **'LARGE'** architecture supporting up to 512 KB RAM - a significant boost over the 1st generation architecture memory constraints. Besides the support for larger programs and the over the air upgrade capability the **'LARGE'** and **'SMALL'** architecture shared the same virtual-machine architecture and did not offer sophisticated features such as multi-threading etc.

In **2006** the **third generation** Execution Architecture was offered named the **X32 architecture**. With the introduction of the **X32 architecture** the RTCU platform was ported to a powerful 32-bit ARM7 architecture and offered sophisticated multi-threading and the support for file-systems - including SD-CARD. Also introduced were advanced compiler and execution optimization technologies further boosting the power and performance of the RTCU devices.

The **fourth generation** Execution Architecture were introduced in **2014** named the **NX32 architecture**.
Besides offering much larger programs exploiting more memory, more threads, more strings, larger channels, etc the **NX32 architecture** is basically a **paradigm change** in the way a project is handled by the RTCU device.
With the **NX32 architecture** the concept of an Intellisync Project Drive is introduced, meaning that all elements of a project is represented by files in a **Intellisync** based file-system.

Finally, in **2017** the **NX32L Architecture** was introduced representing a massive leap in bringing the most sophisticated and open platform for tomorrows demanding IIoT/M2M applications. The **NX32L Architecture** rests on a powerful new hardware and software platform embracing a broad range of crucial technologies in the field of security, wireless communication standard and expandability. The **NX32L Architecture** is operating under a full and highly optimized Linux implementation.

Every new **RTCU Execution Architecture** introduced adhere to the RTCU Compatibility Manifesto.

### 1.2.2 The RTCU Compatibility Manifesto

To ensure device and application longevity the following compatibility manifesto has been formulated more than a **two decades** ago:

***"A given Execution Architecture generation must be binary compatible for a minimum one generation backwards.***
***Additionally full source code compatibility for minimum two generations backwards must be ensured"***

This means that for the **NX32** Execution Architecture it is possible to execute binary **X32** programs and with only re-compilation and no source code changes to execute **LARGE** architecture applications.

The RTCU Compatibility Manifesto has turned out to be a crucial element in the success of the the RTCU concept for more than 20 years - as well as for the future of the concept now transitioning into the exciting era of the **Internet of Things**.


© 2025 Logic IO, www.logicio.com

### 1.2.3 Intellisync Project Drive

Introduced in the NX32 Execution Architecture a new concept for storing and maintaining all elements of a project such as application and voice data - the Intellisync Project Drive - is introduced.

The Project Drive has support for storing not just the application and voice files, but also has support for additional read-only files, e.g. for use as configuration files. All of these files are stored on a single drive, allowing for the most flexible memory usage, where e.g. a small application can be combined with a large amount of voice data, or a very large application could use most of the space for itself.

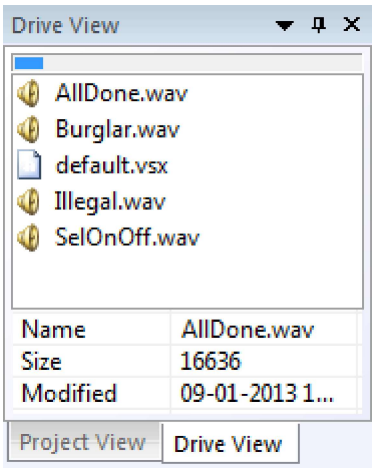
When accessed from application or the file-system window , it appears as a normal, write protected drive, using the drive letter P:, where all the ordinary file system functions work. The Project Drive does not support read out of VSX files.

The Project Drive offers a read only file system including application and voice data.

**Limitations and capabilities:**
NX32 Execution Architecture :
Support for a maximum of 127 files using 8.3 file names and no folders.

NX32L Execution Architecture :
Files using 8.3 file names and folders are supported with drag and drop capabilities. Supports a maximum of 1023 files and folder names are limited to 8 characters of length.

Maintaining the Project Drive is performed using the RTCU IDE or the RTCU Deployment Server utilizing a sophisticated synchronization method.
When the application is built from the RTCU IDE, it creates an .RPC file - a so called RTCU Project Container. This file contains all of the files in the project drive, including the application and the voice files. The .RPC file format is based on standard formats, allowing for files to be changed, added and removed after it has been built, e.g. by using RPCTOOL.EXE.




**Intellisync**

Maintaining the Project Drive is performed using a state-of-the-art **Intellisync** compaction and differencing algorithm that ensures that Project Drive synchronization is performed with minimum overhead and bandwidth usage.

In the **Intellisync** of a project with an RTCU device, the individual files are compared dynamically, and only the difference is transferred.

This means that if a single configuration file is updated on a Project Drive with many large files, only the changed file is transferred and updated, reducing the time and data needed to update it. When the size of a file increases, or a new file is added, there is a risk that it is necessary to move some of the other files to make room for it. This will be more likely to happen on a nearly full Project Drive.


© 2025 Logic IO, www.logicio.com

### 1.2.4 NX32 Execution Architecture

Besides the Intellisync Project Drive the **NX32 Execution Architecture** offers considerable more resources available to the application.

Available application RAM is doubled from **192 KB to 386 KB** and at the same time application size is increased to **1 MB**.
Size of local variables in a FUNCTION is doubled from **256 to 512 bytes,** and there is considerable larger STRING resources available.

The NX32 run-time offers a boost in the multi-threading capabilities with an increase on the number of threads from 14 to 20.
A number of resources, such as the maximum number of threads, MUTEX, SEMAPHORE and CHANNEL are doubled in size/capacity.
Please refer to the RTCU multithreading data sheet for detailed information.

### 1.2.5 NX32L Execution Architecture

The NX32L Execution Architecture operates under Linux and is available on the latest generation of Linux based RTCU devices such as the RTCU NX and RTCU LX series.
NX32L inherits all the features and is fully backward compatible with the NX32 Execution Architecture, This means that any binary executable will run without any changes.

**Using the NX32L compilation mode a long list of enhancements will be available:**

- Support for Instrumented Execution (RTCU IEX).
- VPL application resource enhancements:
  - Image size increased from 1 Mbyte (NX32) to 4 Mbyte.
  - RAM size increased from 390 Kbyte (NX32) to 4 Mbyte.
  - Code size increased from 1 Mbyte (NX32) to 2 Mbyte.
- Project drive enhancements:
  - Multi-level sub-directory support with up to 1023 files.
  - The capacity is increased from 1 Mbyte to 2 Mbyte.
- Large String Support (LSS) is implicit and can not be disabled..
- Non-initialized application variables will not take up space in the execution image. This enables large data-structures without an increase in application size.
- Large array support with an unlimited number of elements and an unlimited size of each element. Only limitation is the RAM size.

**The following run-time resource enhancements are also available:**

- Total number of dynamic strings increased from 600 to 4096.
- Total maximum size of strings increased from 128 Kbyte to 512 Kbyte.
- Number of threads increased from 20 to 64.
- Number of Semaphores increased from 64 to 256.
- Number of Mutexes increased from 64 to 256.
- Number of Channels increased from 32 to 128.
- Total channel memory size increased from 39 Kbyte to 256 Kbyte.
- VPL thread stack size increased from 1 Kbyte to 4 Kbyte.
- Function local variables size increased from 0.5 Kbyte to 1 Kbyte.
- Prepared for VPL application dynamic memory management.

The RTCU NX32L execution architecture was introduced in **Runtime-firmware V1.40.00**.


© 2025 Logic IO, www.logicio.com

### 1.2.6 EIS3

**EIS3 (Enhanced Instruction Set V3)** was introduced in **X32/NX32 firmware V5.00** and NX32L run-time-firmware **V1.30.00** and adds the following functionality:

- Unsigned integers with the following new datatypes: BYTE, USINT and UINT.
- Double floating-point precision support with the new DOUBLE datatype and a dual precision Math library.
- Large String Support (LSS) with up to 16384 characters dynamic STRING length and up-to 1024 characters static strings.

When any of the above features are used in a project it will automatically require EIS3. This information is available in the Project Information dialog.

## 1.3 RTCU Security features

### 1.3.1 RTCU Security Features

The security features are a collection of features in the RTCU devices that can be used to prevent unauthorized access to the device as well as protecting the data and communication against eavesdropping and tampering.

**Network security with TLS**
For NX32L devices, it is possible to enable network security through the use of the Transport Layer Security (TLS) protocol and X509 certificates.
The TLS protocol enables Client-Server applications to use encryption when communicating across a network.
TLS v1.2, v1.1 and 1.0 are supported

The SMTP, MQTT and raw TCP/IP sockets can be configured to use TLS.

**Encrypted RTCU Communication Hub communication**
TLS is fully supported using the RTCU Communication Hub and is the preferred approach for safe communication.
For non NX32L devices the communication can be encrypted by configuring an encryption key with the RTCU Communication Hub Settings dialog or sockSetGWParm.

**Data encryption**
Data encryption is achieved with the Advanced Encryption Standard (AES).
The AES is a specification for the encryption of electronic data established by the U.S. government.
A key length of 128 bit is supported. See rdEncrypt128 and rdDecrypt128 for more information.

**Encrypted include files**
To make it possible to share a parts of a project without providing access to the actual code, encrypted include files can be used.

**Access restriction**
It is possible to use passwords to restrict the access to the RTCU device.
Without the correct password, only type of device, the firmware version, serial number and debug messages can be read from the device.
For more information about passwords, see boardSetPassword and boardSetPasswordAlt.


© 2025 Logic IO, www.logicio.com